Week 13

ID1063 Introduction to Programming in C

Lecture 24: Recursion Trees, Structures+recursion

Recap: Recursion

- Recursion:
  - Express the problem as a base case, plus a call on a smaller problem.
  - Factorial(n) = n * Factorial(n-1)
- `factorial` makes **one** recursive call, so the calls form a *chain*.




```
int fact(int n)
{
    if (n == 0)
        return 1;
    return n * fact(n - 1);
}
```




```
fact(3) -> fact(2) -> fact(1) -> fact(0)
```

Recap: Recursion to find the max

- Given an array `a[0,…,n-1]`, find the maximum

- Recursive formulation:
  - max(a[0…n-1]) = max(a[n-1], a[0,…n-2])
  - Base case: if n=1, return a[0]
- Actual function:
  - `int maximum(int *a, int n)`

- What if we formulate it as :
  - max(a[0…n-1]) = max(a[0], a[1,…n-1])
  - Does the function/code change?

More recursion

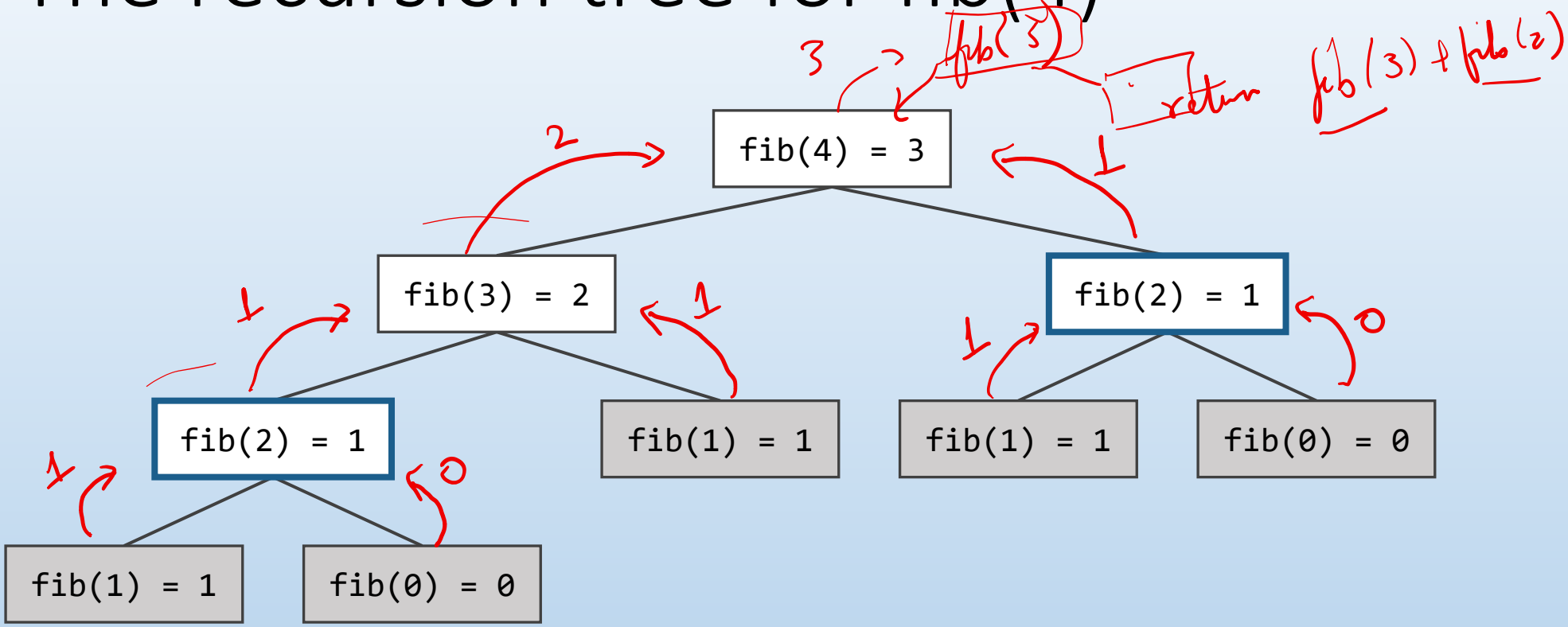
Fibonacci numbers

- 0, 1, 1, 2, 3, 5, 8, 13, 21, … each number is the sum of the previous two.
- fib(0) = 0, fib(1) = 1, fib(n) = fib(n − 1) + fib(n − 2)

```
int fib(int n)
{
    if (n <= 1)                         /* n is 0 or 1 */
        return n;
    return fib(n - 1) + fib(n - 2);     /* two calls */
}
```

- Note the similarity to the definition! Recursive functions are often easy to understand/write.

The recursion tree for fib(4)



- Each call makes two calls, so the calls form a **tree**
- `fib(2)` (outlined) is computed twice: `fib(4) makes 9 total calls to fib.`

How big does the tree get?

- Count the calls (nodes in the tree) made by `fib(n)`:

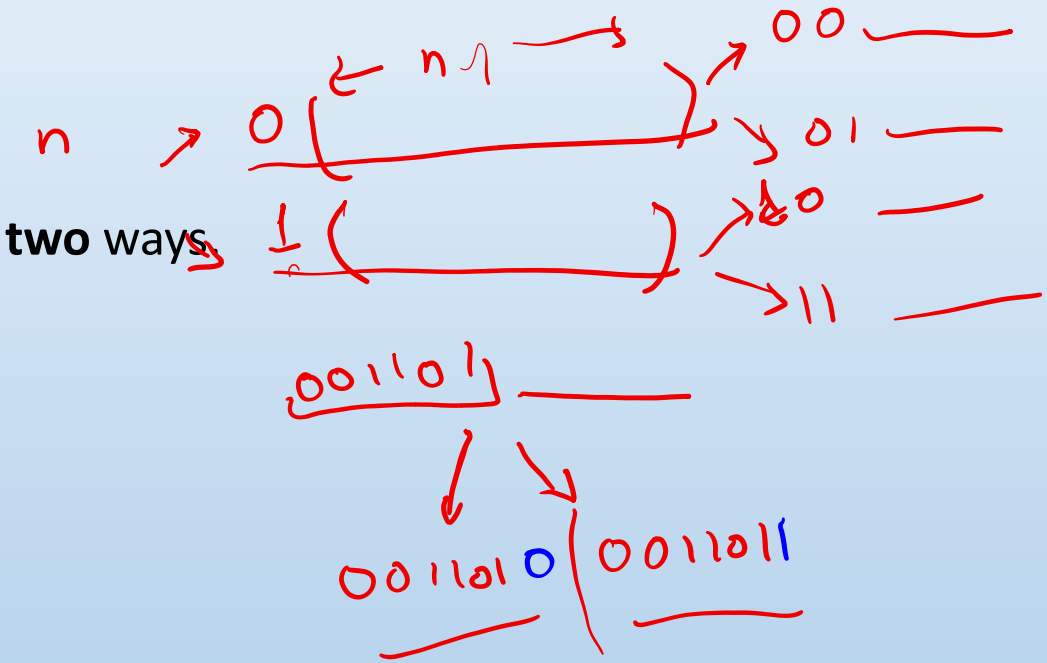
n	fib(n)	Calls made
10	55	177
20	6,765	21,891
30	832,040	2,692,537
40	102,334,155	331,160,281

- A simple loop needs only about n steps.

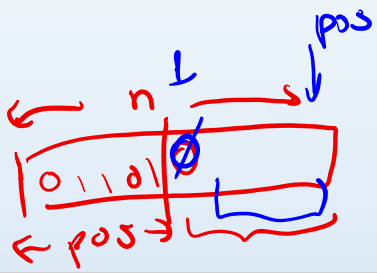
Example 2: Print all binary strings

- Write a function that prints every string of length n made of 0s and 1s.

- Recursive formulation:
  - Fix a prefix. Choose the *next* character in **two** ways.



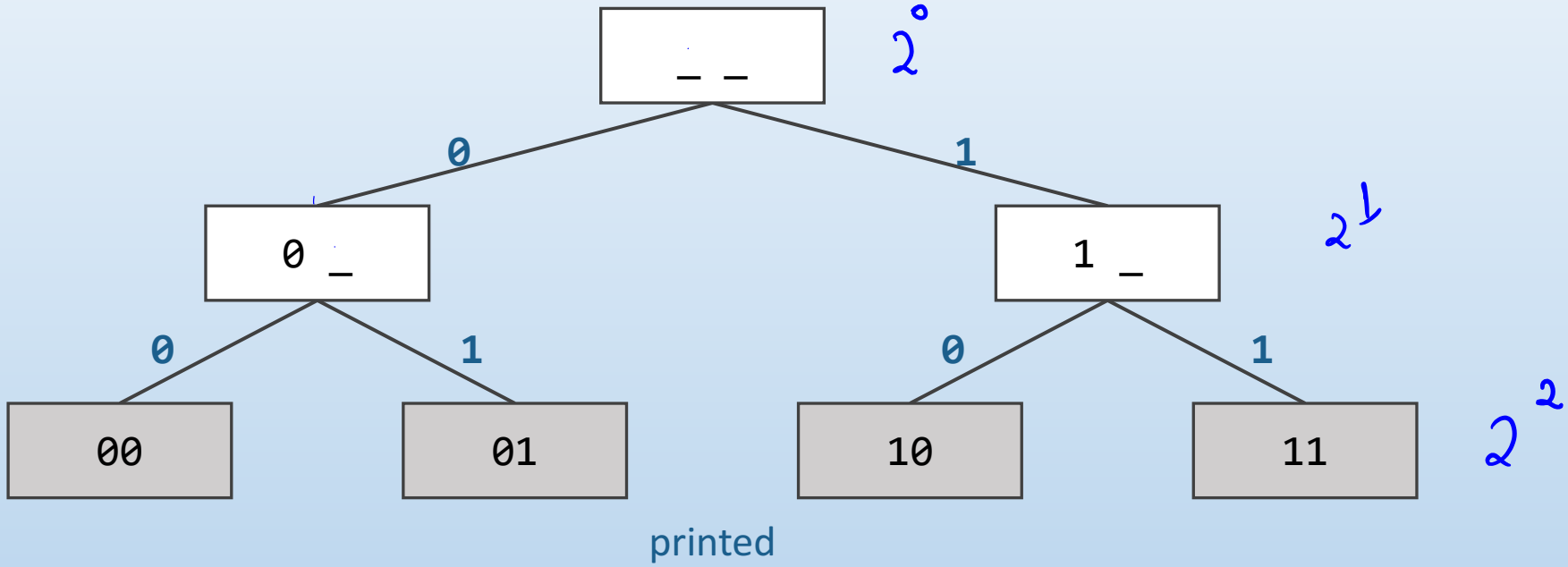
Code




```
void binary(char *s, int pos, int n)
{
    if (pos == n) {                 /* string complete */
        s[n] = '\0';
        printf("%s\n", s);
        return;
    }
    s[pos] = '0';
    binary(s, pos + 1, n);          /* branch 1 */
    s[pos] = '1';
    binary(s, pos + 1, n);          /* branch 2 */
}
```

- `char s[4];`
- `binary(s, 0, 3);` /*prints 000, 001, …, 111. */

The tree for binary(s, 0, 2)

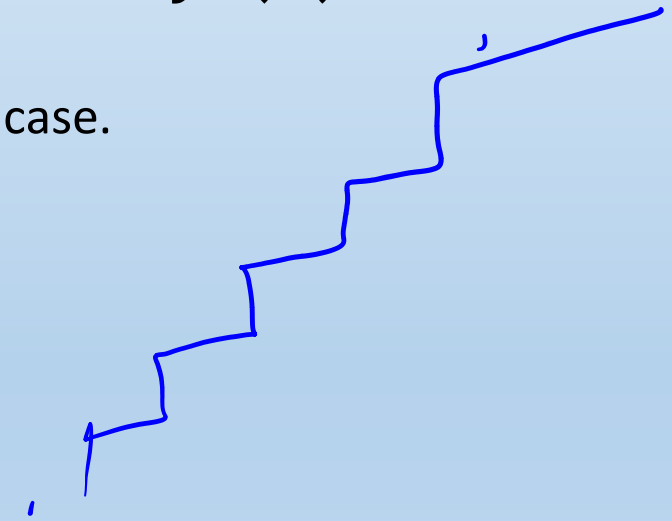


- Each level fixes one more character
- Each leaf is one printed string.
- Length n gives 2 × 2 × … × 2 (n times)=2^n leaves: the tree doubles at every level.

Questions

- For `binary(s, 0, 3)`:
  - How many strings are printed?
  - How many calls to `binary` are made in total?

- You can climb stairs 1 or 2 steps at a time. Let `ways(n)` be the number of ways to climb n stairs.
  - What are the base cases? Write the recursive case.



Structures that point to their own type

Can a structure contain itself?

- A person has a name and a contact parent, who is also a person.

```
struct Person {
    char name[20];
    struct Person parent;    /* a Person inside? */
};
```

- Q: What would `sizeof(struct Person)` be?



- The compiler cannot know the size of this struct! Error.

A self-referential structure

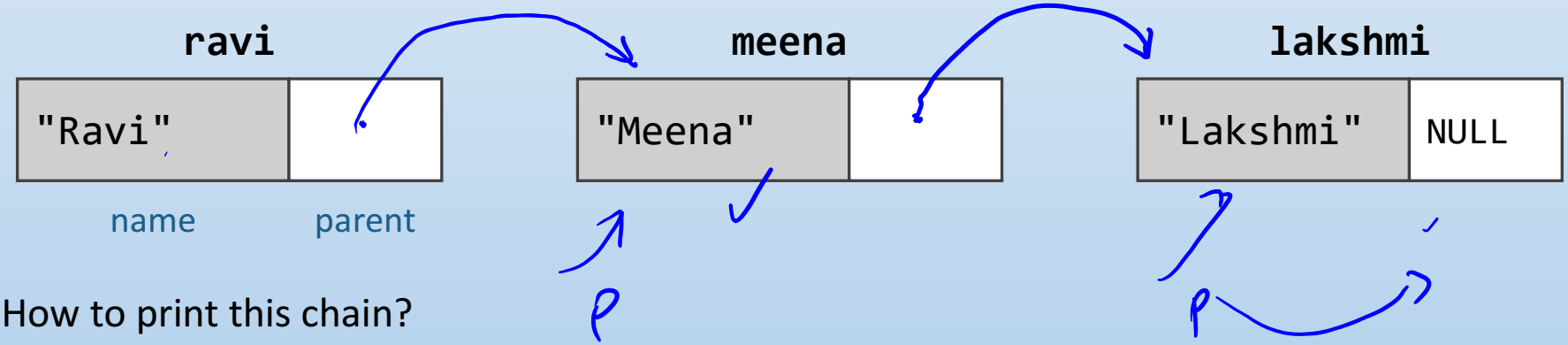
- A pointer has a fixed size (8 bytes), whatever it points to.
- So a person can store the **address** of their contact parent.

```
typedef struct Person {
    char name[20];
    struct Person *parent;    /* pointer: OK */
} Person;
```

- The tag `struct Person` is needed: inside the braces, `Person` is not yet defined.
- `parent == NULL` means: no parent recorded. The chain ends there.

A chain of structs

```
Person lakshmi = {"Lakshmi", NULL};
Person meena   = {"Meena", &lakshmi};
Person ravi    = {"Ravi", &meena};
```



How to print this chain?

```
 Person *p = &ravi;
while (p != NULL) {
   printf("%s\n", p->name);      /* Ravi Meena Lakshmi */
   p = p->parent;}
```

Recursion on the chain

- A chain is either empty (NULL), or a person followed by a chain.
- Q: Given a pointer to a person, determine his/her generation

```
int countgenerations(Person *p)
{
    if (p == NULL)                    /* empty chain */
        return 0;
    return 1 + countgenerations(p->parent);
}

printf("%d\n", countgenerations(&ravi));     /* 3 */
```

- Base case: NULL, recursive call on `p->parent`.

- One pointer to follow, so the recursion structure is a *chain* (like factorial)

Print before or after the call?

Two ways to *recursively* print the chain of people

Print, then recurse

```
void youngestFirst(Person *p)
{
    if (p == NULL)
        return;
    printf("%s\n", p->name);
    youngestFirst(p->parent);
}
```

Prints: Ravi, Meena, Lakshmi

Recurse, then print

```
void oldestFirst(Person *p)
{
    if (p == NULL)
        return;
    oldestFirst(p->parent);
    printf("%s\n", p->name);
}
```

Prints: Lakshmi, Meena, Ravi

- Q: Why is the order reversed?
- A: Either do own work and then call subroutines, or call and then do own work.

Creating structures on the heap

```
Person *makePerson(const char *name, Person *parent)
{
    Person *p = malloc(sizeof(Person));
    if (p == NULL)
        return NULL;
    strcpy(p->name, name);      /* library call */
    p->parent = parent;
    return p;
}

void freeChain(Person *p)
{
    if (p == NULL)
        return;
    freeChain(p->parent);       /* older ones first */
    free(p);
}
```

Optional: a family tree

- With **two** self-pointers, the data itself is shaped like a tree.

```
typedef struct Member {
    char name[20];
    struct Member *mother, *father;
} Member;

int countPeople(Member *m)      /* m + ancestors */
{
    if (m == NULL)
        return 0;
    return 1 + countPeople(m->mother)
             + countPeople(m->father);
}
```

- CountPeople recursively counts the number of people in the family connected to m (as an ancestor)
- Two pointers to follow, so two calls: the recursion tree **is** the family tree.

File input and output

(streams)

Streams you already know

- A **stream** is a sequence of characters flowing into or out of a program.
- Three streams are open when every C program starts:

Stream	Connected to	Used by
`stdin`	keyboard	`scanf`, `getchar`
`stdout`	screen	`printf`, `putchar`
`Stderr`	screen (for errors)	`perror`

- A *file* is just data stored on disk (as a string)
- A file opened with `fopen` is just one more stream, of type `FILE *`.
- Writing to /reading from a file: `fprintf`, `fscanf`, `fgets`.

Opening and closing a file

```
FILE *fp = fopen("class.txt", "r"); /*reading mode*/
if (fp == NULL) {
    printf("Error opening class.txt");  /* prints the reason */
    return 1;
}
/* ... read from fp ... */
fclose(fp);
```

Mode	Meaning	If the file exists	If not
"r"	read	read from the start	fails: NULL
"w"	write	**erases** it first	creates it
"a"	append	adds at the end	creates it

FILE is internally a typedef'ed struct that is a file stream.
Contains state of read, current position in stream, etc.

Writing to a file: fprintf

- `fprintf` is `printf` with a stream as its first argument.

```
FILE *out = fopen("report.txt", "w");
if (out == NULL)
    return 1;

fprintf(out, "Roll Name Marks\n");
fprintf(out, "%d %s %.1f\n", s.roll, s.name, s.marks);
fclose(out);
```

- `printf(...)` is the same as `fprintf(stdout, ...)`.
- Open `report.txt` to see the output.

Reading with fscanf

- `fscanf` is `scanf` reading from a stream.
- It returns how many items it read: stop when that is not what you asked for.

```
Student s;
while (fscanf(fp, "%d %19s %lf",
              &s.roll, s.name, &s.marks) == 3)
    printf("%s has %.1f\n", s.name, s.marks);
```

- At the end of the file it returns **`EOF (-1)`**
- On a badly formatted line, returns 0.

Reading a line: fgets

```
char *fgets(char *buf, int size, FILE *stream);
```

- Reads up to `size - 1` characters, and stops after a newline `'\n'`.
- Keeps the `'\n'`, adds `'\0'`; returns `NULL` at the end of the file.
- Never writes past `buf`: the size is part of the call.

```
char line[100];
while (fgets(line, sizeof line, fp) != NULL)
    printf("Read: %s", line);     /* line already ends in \n */
```